Object Oriented Programming (CSEG1044)

Unit 05 – Lecture 01: Collections Framework Overview + Iteration + List

April 1, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Collections Framework and Wrapper Classes

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Collections Framework and Wrapper Classes
- **Course thread:** Use Collections Framework Overview + Iteration + List to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Recall Before You Start

- **Generics**: Collections are safer and clearer once typed containers are already familiar. Main reference: `Unit 04 / Lecture 01 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Recall Before You Start

- **Generics**: Collections are safer and clearer once typed containers are already familiar. Main reference: `Unit 04 / Lecture 01 slides/notes`.
- **Comparison logic**: Sorting becomes easier once small comparison behavior is familiar. Main reference: `Unit 04 / Lecture 03 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Learning Outcomes

- Explain `Iterable`, `Collection`, and `Iterator`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Learning Outcomes

- Explain `Iterable`, `Collection`, and `Iterator`.
- Choose `ArrayList` vs `LinkedList` for common use cases.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Learning Outcomes

- Explain `Iterable`, `Collection`, and `Iterator`.
- Choose `ArrayList` vs `LinkedList` for common use cases.
- Sort a list using `Comparable` and `Comparator` (with lambdas).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Terms to Know

- **Collection**: group of elements (root interface for many collections).
- **Iterable**: can be used in for-each loop (provides `iterator()`).
- **Iterator**: cursor used to traverse a collection safely.
- **List**: ordered collection with duplicates allowed and index-based access.
- **Comparable**: natural ordering (`compareTo`).
- **Comparator**: custom ordering (`compare`).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Core interfaces: `Iterable`, `Collection`, `Iterator`* → *List: ArrayList vs LinkedList* → *Sorting: Comparable vs Comparator*.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Core interfaces: `Iterable`, `Collection`, `Iterator`* → *List: ArrayList vs LinkedList* → *Sorting: Comparable vs Comparator*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Core interfaces: `Iterable`, `Collection`, `Iterator`* → *List: ArrayList vs LinkedList* → *Sorting: Comparable vs Comparator*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Unit 05: Collections and Wrapper Class

- Lecture focus: Collections Framework Overview + Iteration + List

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Today: Roadmap

- Core interfaces: ‘Collection‘, ‘Iterable‘, ‘Iterator‘

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Today: Roadmap

- Core interfaces: ‘Collection‘, ‘Iterable‘, ‘Iterator‘
- List interface; ‘ArrayList‘ vs ‘LinkedList‘ (when to use which)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Today: Roadmap

- Core interfaces: ‘Collection‘, ‘Iterable‘, ‘Iterator‘
- List interface; ‘ArrayList‘ vs ‘LinkedList‘ (when to use which)
- Sorting with ‘Comparable‘/‘Comparator‘ (use lambdas where helpful)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Collections Framework (why)

- Standard data structures + algorithms (reduce bugs).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Collections Framework (why)

- Standard data structures + algorithms (reduce bugs).
- Works well with generics: `List<Student>`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Collections Framework (why)

- Standard data structures + algorithms (reduce bugs).
- Works well with generics: `List<Student>`.
- Includes sorting utilities and iterators.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Iteration (safe removal)

- for-each loop uses an `Iterator` internally.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Iteration (safe removal)

- for-each loop uses an `Iterator` internally.
- Removing inside for-each can throw `ConcurrentModificationException`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Iteration (safe removal)

- for-each loop uses an `Iterator` internally.
- Removing inside for-each can throw `ConcurrentModificationException`.
- Use `Iterator.remove()` when you need to delete while looping.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

ArrayList vs LinkedList (quick)

- `ArrayList`: fast random access; best default choice.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

ArrayList vs LinkedList (quick)

- `ArrayList`: fast random access; best default choice.
- `LinkedList`: slower random access; useful as queue/deque.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Sorting: Comparable vs Comparator

- `Comparable`: natural ordering (`compareTo`) in the class.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Sorting: Comparable vs Comparator

- `Comparable`: natural ordering (`compareTo`) in the class.
- `Comparator`: custom ordering (`compare`) outside the class.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Sorting: Comparable vs Comparator

- `Comparable`: natural ordering (`compareTo`) in the class.
- `Comparator`: custom ordering (`compare`) outside the class.
- Lambdas make comparators short and readable.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *Collections Framework Overview + Iteration + List* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Core interfaces: `Iterable`, `Collection`, `Iterator`*, *List: ArrayList vs LinkedList*, the worked example, or the recap?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Example: List + iterator + sorting (1/2)

```
import java.util.*;

class Student {
    final String rollNo;
    final double cgpa;
    Student(String rollNo, double cgpa) { this.rollNo = rollNo;
        this.cgpa = cgpa; }
    double getCgpa() { return cgpa; }
    public String toString() { return rollNo + ":" + cgpa; }
}

public class Demo {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("21CS1002", 9.1));
        list.add(new Student("21CS1001", 8.2));
        list.add(new Student("21CS1003", 3.9));
```

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Example: List + iterator + sorting (2/2)

```
// continued...
        Iterator<Student> it = list.iterator();
        while (it.hasNext()) {
            if (it.next().getCgpa() < 4.0) it.remove();
        }

        list.sort((a, b) -> Double.compare(b.getCgpa(), a.
            getCgpa())); // desc
        System.out.println(list);
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Why use `Iterator.remove()` when deleting during traversal?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Why use `Iterator.remove()` when deleting during traversal?
2. Which list is best for random access: `ArrayList` or `LinkedList`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Why use `Iterator.remove()` when deleting during traversal?
2. Which list is best for random access: `ArrayList` or `LinkedList`?
3. Difference between `Comparable` and `Comparator`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Watch-outs

- Ignoring generics and using raw collections.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Watch-outs

- Ignoring generics and using raw collections.
- Changing a collection unsafely while iterating.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Watch-outs

- Ignoring generics and using raw collections.
- Changing a collection unsafely while iterating.
- Choosing a List when the real requirement is uniqueness.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Takeaways

- Lists store ordered data and allow duplicates.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Takeaways

- Lists store ordered data and allow duplicates.
- Iterators enable safe traversal and removal.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Takeaways

- Lists store ordered data and allow duplicates.
- Iterators enable safe traversal and removal.
- Sorting is done using `Comparable` or `Comparator`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint Questions

1. What is the purpose of `Iterator`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint Questions

1. What is the purpose of `Iterator`?
2. When do you choose `LinkedList` over `ArrayList`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint Questions

1. What is the purpose of `Iterator`?
2. When do you choose `LinkedList` over `ArrayList`?
3. How do you sort a list by a custom rule?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Lecture Preview

Next: Set/SortedSet + Map/SortedMap

- Uniqueness (Set) and key-value storage (Map) with hashing basics.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).